

从 C++ 到 AI 计算：第二册

SIMD、矩阵乘、卷积、Attention、量化与高性能推理

CPU Performance Study

本册接续第一册的机器执行模型，进入现代 CPU 上的 AI 计算和高性能工程。正文以 LaTeX 正式书稿为准，PDF 和 EPUB 使用同一份源文件生成。

前言

第一册把一个 C++ 程序怎样变成进程、指令、寄存器、栈帧、二进制接口和可测量的性能现象讲清楚。第二册从这里继续往前走：当程序不再只是执行几条整数指令，而是在 CPU 上做矩阵乘、卷积、归一化、Softmax、Attention、量化和端到端推理时，性能问题会从“这一行代码慢不慢”变成“计算量、内存流量、数据布局、向量宽度、缓存层级、线程切分和数值误差如何同时约束一个系统”。

本册的核心目标不是教读者背几个优化技巧，而是建立一套稳定的分析方法。看到一个 AI 算子时，先问它做了多少乘加，读写了多少字节，复用发生在哪里，瓶颈更像计算吞吐还是内存带宽；再问标量循环怎样映射到 SIMD lane，分块怎样把数据留在缓存里，packing 为什么会增加一次拷贝却减少大量随机访问，量化为什么会改变瓶颈，线程池为什么有时加线程反而变慢。只有这些问题能被逐层回答，性能工程才不是碰运气。

本册会继续保持第一册的写法：原理先行，代码只是证据；实验必须解释输入、环境和误差；源码阅读不追求把一个项目从头背下来，而是抓住关键数据结构和关键循环。读者读完后应该能独立拆解一个简单推理算子，能写出可信 benchmark，能解释矩阵乘和 Attention 的主要性能来源，也能读懂 oneDNN、OpenBLAS、XNNPACK、llama.cpp 这类工程里的核心结构。

核心思想

第二册的主线可以压缩成一句话：AI 推理性能不是某个孤立函数的速度，而是计算图、数值格式、内存层级、SIMD 指令和线程运行时共同形成的吞吐系统。

本册不是第一册的重复。第一册解决“机器如何执行程序”，第二册解决“怎样把大量数值计算安排成机器愿意高速执行的形状”。前者让读者看懂汇编和 ABI，后者让读者看懂高性能内核、推理框架和系统级性能报告。

怎样阅读本册

本册适合已经读过第一册，或者至少知道进程、虚拟内存、缓存、汇编、ABI、benchmark 基础概念的读者。没有这些基础也可以读，但遇到寄存器、cache line、TLB、调用约定、perf 计数器时，要回到第一册补齐背景。

阅读顺序建议按章节推进。第一部分先建立性能模型，因为没有模型就无法判断一个优化是否值得做；第二部分讨论内存层级和 SIMD，因为现代 CPU 的浮点吞吐通常远高于内存能提供数据的速度；第三部分进入矩阵乘、packing、卷积和数据流，这是大多数 AI 计算库的核心；第四部分进入 Transformer 推理，把前面的模型放进 Softmax、Attention 和 KV Cache；第五部分讲量化、线程运行时和源码阅读，帮助读者把局部算子放进完整推理系统。

每章阅读时可以按四步做笔记。第一步写下本章处理的核心矛盾，例如“计算多但是数据复用差”或“单核快但是多线程抢内存”。第二步写下最朴素的实现，因为朴素实现通常暴露了真实瓶颈。第三步写下优化把什么成本换成了什么收益，例如 packing 用顺序拷贝换缓存命中，SIMD 用更宽的寄存器换更严格的数据排列。第四步写下可以测量的指标，例如 cycles、instructions、cache misses、bandwidth、GFLOP/s、tokens/s 和 P95 latency。

常见误区

不要把本册当成“优化技巧清单”。技巧只有在模型里才有意义。比如循环展开、预取、对齐、量化、线程绑定、算子融合都可能有用，也都可能让程序更慢。判断依据不是经验口号，而是瓶颈位置、数据规模和测量证据。

代码示例主要用 C++20 表达。示例偏教学，不追求覆盖所有平台指令集；涉及平台相关 SIMD 时，会先讲清楚标量语义和向量语义，再讨论 AVX2、AVX-512、NEON 或库内核如何落地。

全书结构

本册分为五个部分。

第一部分是性能模型。它解释 AI 计算为什么不能只看时间，必须同时看操作数、字节数、复用、Roofline、基准测试设计和计数器证据。这个部分为后面所有优化建立共同语言。

第二部分是内存与 SIMD。它从 cache line、TLB、预取、AoS/SoA 数据布局讲到标量循环如何变成向量循环。读者需要在这里形成一个基本判断：CPU 的峰值吞吐很高，但前提是数据能以合适的形状到达执行单元。

第三部分是张量内核。矩阵乘、分块、packing、微内核、卷积和算子融合是高性能 AI 计算的核心。这里会从三重循环开始，一步一步解释为什么工业库会把简单公式拆成看似复杂的层级结构。

第四部分是 Transformer 推理。Softmax、LayerNorm、Attention 和 KV Cache 把数值稳定性、内存流量和计算图调度放在同一个问题里。读者将在这里看到“算得快”和“少搬数据”怎样互相制约。

第五部分是量化、并行运行时和源码阅读。量化改变了数据宽度和误差模型，线程运行时决定多核扩展，源码阅读和 benchmark suite 则决定我们能否把局部知识变成可维护的工程能力。

心智模型

本册的结构是一条从底到上的链：性能模型给出判断标准，内存和 SIMD 给出单核执行能力，张量内核给出高复用计算形状，Transformer 给出真实 AI 推理负载，量化和运行时把算子放进端到端系统。

目录

第一部分	性能模型：从指令到吞吐	
第 1 章	AI 计算在 CPU 上的基本地图	2
第 2 章	Roofline 模型与可信测量	5
第二部分	内存与 SIMD：让数据喂饱执行单元	
第 3 章	缓存、TLB、预取与数据布局	8
第 4 章	SIMD：从标量循环到向量化	10
第三部分	张量内核：矩阵、卷积与算子融合	
第 5 章	矩阵乘：从三重循环到性能结构	13
第 6 章	分块、Packing 与微内核	16
第 7 章	卷积、数据流与算子融合	19
第四部分	Transformer 推理：从计算图到内存流	
第 8 章	归一化、Softmax 与数值稳定性	23
第 9 章	Attention、KV Cache 与内存流量	26
第五部分	量化、并行运行时与源码阅读	
第 10 章	量化与低精度计算	29
第 11 章	线程池、任务切分与 NUMA	31
第 12 章	源码阅读、Benchmark Suite 与工程报告	33
附录 A	实验路线	35
附录 B	源码阅读索引	36

第零部分

性能模型：从指令到吞吐

第 1 章

AI 计算在 CPU 上的基本地图

AI 计算看起来像一串数学公式，落到 CPU 上却是一串非常具体的动作：从内存读参数和激活值，把数据搬进缓存和寄存器，执行乘法、加法、比较、指数、归约，再把结果写回内存。理解这张地图的目的，是避免把“模型很大”“矩阵很多”“代码用了 SIMD”这些模糊说法混在一起。真正能指导优化的，是每一层到底做了多少运算、移动了多少数据、复用了多少次、受哪个硬件资源限制。

1.1 从模型公式到机器动作

以最常见的线性层为例，数学上它写成 $Y = XW + b$ 。这个公式很短，但 CPU 看到的不是矩阵符号，而是循环、地址、加载、乘加和存储。假设 X 是一批输入， W 是权重矩阵， Y 是输出矩阵，那么每一个输出元素都要遍历一整段输入和权重。朴素代码会把这个过程写成三重循环：外层枚举输出行，中间枚举输出列，内层做点积。

Listing 1.1 线性层的朴素三重循环

```
1 #include <cstddef>
2 #include <cstdint>
3 #include <span>
4
5 void linear_naive(std::span<const float> x,
6                  std::span<const float> w,
7                  std::span<const float> bias,
8                  std::span<float> y,
9                  std::int32_t batch,
10                 std::int32_t input_dim,
11                 std::int32_t output_dim) {
12     for (std::int32_t row = 0; row < batch; ++row) {
13         for (std::int32_t col = 0; col < output_dim; ++col) {
14             float acc = bias[static_cast<std::size_t>(col)];
15             for (std::int32_t k = 0; k < input_dim; ++k) {
16                 const std::size_t x_index =
17                     static_cast<std::size_t>(row) * input_dim + k;
18                 const std::size_t w_index =
19                     static_cast<std::size_t>(k) * output_dim + col;
20                 acc += x[x_index] * w[w_index];
21             }
22             y[static_cast<std::size_t>(row) * output_dim + col] = acc;
23         }
24     }
25 }
```

这段代码的教学价值不在于快，而在于它把问题拆开了。内层循环每走一步，需要读一个输入元素、读一个权重元素、做一次乘法和一次加法。输出元素只有最后写一次，偏置只读一次。如果输入维度很大，那么主要成本来自内层循环。于是第一个问题不是“能不能优化”，而是“内层循环的瓶颈是什么”。如果权重访问跨步很大，缓存命中会差；如果输入被多个输出列重复使用却每次重新加载，复用就没有被充分利用；如果编译器无法向量化，浮点执行单元就吃不满。

心智模型

AI 算子首先是数据流，其次才是公式。公式告诉我们结果是什么，数据流告诉我们机器为了得到结果必须搬多少数据、保留哪些中间值、在哪些层级复用。

1.2 操作数、字节数和复用

高性能计算里常用两个基本量描述一个算子。第一个是操作数，通常用 FLOPs 表示浮点操作数量。一次乘加有时按两个浮点操作计数，因为它包含一次乘法和一次加法；使用 FMA 指令时，硬件可能一条指令完成乘加，但数学工作量仍然按两个操作理解。第二个是字节数，也就是为了完成这些操作至少要从内存层级搬动多少数据。操作数决定计算压力，字节数决定带宽压力。

仅有操作数还不够。一个矩阵乘的 FLOPs 很高，但如果每个权重元素被大量输出复用，内存流量可以被摊薄；一个逐元素激活函数的 FLOPs 不高，但每个元素通常只读一次写一次，复用极少，容易变成内存带宽问题。复用是连接操作数和字节数的关键。相同的公式，数据布局不同，复用位置就不同；相同的布局，循环顺序不同，复用被缓存捕获的概率也不同。

可以用一个非常粗的估算理解线性层。若输入是 $B \times K$ ，权重是 $K \times N$ ，输出是 $B \times N$ ，乘加操作约为 $2BKN$ 。最理想情况下，输入、权重和输出各读写接近一次，字节数大致与 $(BK + KN + BN)$ 成正比；最糟糕情况下，权重和输入在缓存里留不住，每个输出元素都重新从较慢层级读取，实际流量会接近操作数规模。高性能内核的很多复杂结构，本质上就是让实际流量尽量接近理想下界。

核心理想

优化 AI 算子的第一步是写出两个账本：计算账本记录乘法、加法、比较、指数、归约；内存账本记录输入、权重、中间结果和输出的读写字节。没有账本，性能讨论很容易变成猜测。

1.3 算子、内核和端到端推理

讨论 AI 性能时，要区分三个层级。第一层是算子，例如矩阵乘、卷积、Softmax、LayerNorm、Embedding lookup。第二层是内核，也就是某个算子的具体实现，例如 AVX2 的矩阵乘微内核、int8 量化 GEMM、分块 Softmax。第三层是端到端推理，它把多个算子按计算图串起来，还包含内存分配、线程调度、KV Cache 管理、采样和输入输出处理。

一个单独内核很快，不代表端到端推理就快。原因有三类。第一类是形状不匹配，库内核在大矩阵上效率很高，但真实推理可能是 batch 很小、序列长度变化、矩阵很瘦。第二类是中间结果流量，两个算子单独看都快，但中间张量写回内存再读出来，会浪费带宽。第三类是运行时开销，线程池唤醒、任务切分、同步、NUMA 跨节点访问都可能吞掉内核收益。

因此本册不会把“算子优化”和“系统优化”分成互不相干的两件事。我们会先把单个内核讲清楚，再不断问它放进计算图后会发生什么。例如 GEMM 的微内核关注寄存器块，Attention 关注 QK^T 、Softmax 和 PV 之间的中间矩阵，量化关注每个分组的 scale 和 zero point，运行时关注这些任务怎样切给多个核心。

1.4 为什么 CPU 仍然重要

AI 计算常常和 GPU 绑定在一起，但 CPU 仍然重要。第一，很多部署环境没有独立 GPU，或者功耗、成本、内存容量、启动延迟要求使 CPU 更合适。第二，推理系统总有 CPU 侧工作，例如 tokenization、调度、采样、内存管理、IO 和小算子。第三，理解 CPU 上的性能模型能帮助读者理解更一般的硬件原则：数据搬运昂贵，局部性决定效率，向量化需要规则形状，并行扩展受同步和带宽限制。

CPU 的优势是通用、低延迟、控制流强、内存容量大、系统集成度高；劣势是峰值矩阵吞吐通常低于专用加速器。把 CPU 用好，不是把它想象成小 GPU，而是尊重它的结构：少量强核心、复杂缓存层级、宽 SIMD、强分支预测、成熟操作系统调度和丰富性能计数器。第二册的所有章节都会围绕这些结构展开。

习题与作业

选择一个你熟悉的模型层，例如全连接层、卷积层或 Attention 层，写出它的输入、输出和参数形状。先估算 FLOPs，再估算至少需要读写多少字节。不要急着优化，先判断它更像计算受限还是内存受限。

第 2 章

Roofline 模型与可信测量

性能优化最危险的习惯，是看到时间变短就认为自己理解了原因。时间只是结果，不是解释。一个程序变快，可能是指令少了，可能是缓存命中好了，可能是编译器做了新的向量化，也可能只是输入刚好留在缓存里。Roofline 模型提供了一个粗但非常有用的坐标系：横轴看计算强度，也就是每搬一个字节做多少操作；纵轴看实际吞吐；硬件的计算峰值和内存带宽给出上界。

2.1 计算强度的意义

计算强度 (arithmetic intensity) 定义为操作数除以字节数。若一个算子做了很多乘加，但只需要少量数据并能反复复用，那么计算强度高；若一个算子只是读一个元素、做很少计算、写一个元素，那么计算强度低。高计算强度算子更可能被浮点吞吐限制，低计算强度算子更可能被内存带宽限制。

例如向量加法 $C = A + B$ ，每个元素读两个 float、写一个 float，大约移动十二个字节，只做一次加法，计算强度很低。矩阵乘 $C = AB$ 如果分块做得好，每个块里的元素会被重复使用，计算强度可以很高。Softmax 介于两者之间：它有指数、求和和除法，看起来计算不少，但通常需要多次扫描同一行，还涉及数值稳定性和归约，最终瓶颈取决于行长度、实现方式和向量化程度。

深入理解

Roofline 不是精确模拟器。它不会告诉你分支预测、端口压力、TLB miss、依赖链、前端解码和线程同步的全部细节。它的价值在于先排除不可能：如果计算强度很低，而你的吞吐已经接近内存带宽上界，再继续手写 FMA 通常不会带来数量级提升。

2.2 建立测量边界

可信测量要先定义边界。测单个内核时，输入数据是否包含初始化时间，是否包含内存分配，是否包含线程池创建，是否包含随机数生成，都必须说清楚。测端到端推理时，也要区分 prefill、decode、采样、IO、tokenization 和模型计算。边界不同，数字没有可比性。

最小的 benchmark 应该至少记录五件事：机器和编译器信息、输入形状、重复次数、预热策略、计时范围。预热的目的不是“让数字好看”，而是让动态链接、页错误、缓存冷启动、频率变化等一次性成本不污染稳定阶段。重复次数不能机械固定，应当让总运行时间足够长，避免计时器分辨率和调度抖动主导结果。

Listing 2.1 最小计时框架：只计核心循环

```
1 #include <chrono>
2 #include <cstdint>
3 #include <functional>
4 #include <iostream>
5
6 double measure_seconds(const std::function<void()>& work,
7                        std::int32_t warmup,
8                        std::int32_t repeat) {
9     for (std::int32_t i = 0; i < warmup; ++i) {
10         work();
11     }
12
13     const auto begin = std::chrono::steady_clock::now();
14     for (std::int32_t i = 0; i < repeat; ++i) {
```

```

15     work();
16 }
17 const auto end = std::chrono::steady_clock::now();
18 const std::chrono::duration<double> elapsed = end - begin;
19 return elapsed.count() / static_cast<double>(repeat);
20 }

```

这个框架仍然很朴素。真实项目中还要防止编译器把结果优化掉，要固定输入或记录随机种子，要把输出校验和写到不可优化的位置，要记录 CPU 频率策略。若测多线程，还要说明线程数、亲和性、NUMA 节点和是否与系统其他任务竞争。

2.3 计数器证据

时间告诉我们结果，性能计数器帮助解释原因。在 Linux 上常用 perf stat、perf record 和 perf report。对于本册的主题，常看的指标可以分成三类：执行侧看 cycles、instructions、branches 和 branch-misses；缓存侧看 cache-references、cache-misses、L1 data cache load misses、LLC misses 和 dTLB misses；平台相关部分再补充 SIMD 浮点计数器和内存带宽事件。

计数器也有误差。不同 CPU 的事件名不同，虚拟化环境可能屏蔽部分事件，过短程序的采样不稳定，多线程程序的进程级计数可能混入调度影响。因此计数器不能机械崇拜，而要与模型对照。若 Roofline 估算认为程序应受内存限制，而 perf 显示 LLC miss 和带宽很高，这两者互相支持；若计数器与模型冲突，就要检查输入规模、缓存热度、编译选项和测量边界。

实验建议：写三个内核，分别是向量加法、点积和朴素矩阵乘。对每个内核记录运行时间、估算 FLOPs、估算读写字节，并用 **perfstat** 查看 cycles、instructions 和 cache-misses。观察哪个内核更接近内存带宽问题，哪个内核更接近计算吞吐问题。

2.4 从数字回到决策

测量的最终目的不是报一个漂亮数字，而是做决策。若一个矩阵乘只有理论峰值的百分之五，且 cache miss 很高，那么下一步可能是改循环顺序、分块或 packing；若一个逐元素激活已经达到很高内存带宽，那么下一步可能是算子融合，减少中间张量读写；若多线程从四核到八核不再加速，那么下一步要看带宽饱和、任务粒度、同步和 NUMA。

这就是本册后续章节的工作方式。每个技术点都要回到三个问题：它改变了计算账本还是内存账本，它把瓶颈从哪里移动到哪里，它能被什么实验验证。无法回答这三个问题的优化，最多算一次尝试，不能算工程结论。

常见误区

不要只报告最快一次。最快一次通常最不代表真实服务，因为它避开了调度抖动、缓存冷启动和尾延迟。教材中的内核实验可以看平均值，系统实验还要看分位数和稳定性。

第零部分

内存与 SIMD：让数据喂饱执行单元

第 3 章

缓存、TLB、预取与数据布局

CPU 执行单元的速度很快，但它只能处理已经到达寄存器的数据。数据从内存到寄存器之间要经过多级缓存、TLB、加载队列、存储队列和预取器。AI 算子的大部分优化都在处理同一个矛盾：数学上相邻的元素，不一定在内存里相邻；代码里重复使用的值，不一定能在缓存里留到下一次使用。

3.1 缓存层级不是抽象背景

缓存以 cache line 为基本单位搬运数据，常见大小是 64 字节。读取一个 float 时，硬件通常会把同一条 cache line 里的相邻元素一起带进来。如果后续代码顺序访问这些元素，带宽利用率高；如果每次跨很大步长访问，只用到每条 cache line 里的一个元素，实际带宽会被浪费。

矩阵按行主序存储时，连续访问一行是顺序访问，连续访问一列是跨步访问。第一册已经讲过地址计算，第二册要进一步问：这个地址序列对 cache line 是否友好，对硬件预取器是否友好，对 TLB 是否友好。一个三重循环的数学结果不变，但循环顺序不同，内存访问轨迹完全不同。

Listing 3.1 同一个矩阵，行访问和列访问的内存轨迹不同

```
1 #include <cstdint>
2 #include <cstdint>
3 #include <span>
4
5 float sum_by_rows(std::span<const float> a,
6                  std::int32_t rows,
7                  std::int32_t cols) {
8     float acc = 0.0F;
9     for (std::int32_t r = 0; r < rows; ++r) {
10         for (std::int32_t c = 0; c < cols; ++c) {
11             acc += a[static_cast<std::size_t>(r) * cols + c];
12         }
13     }
14     return acc;
15 }
16
17 float sum_by_cols(std::span<const float> a,
18                  std::int32_t rows,
19                  std::int32_t cols) {
20     float acc = 0.0F;
21     for (std::int32_t c = 0; c < cols; ++c) {
22         for (std::int32_t r = 0; r < rows; ++r) {
23             acc += a[static_cast<std::size_t>(r) * cols + c];
24         }
25     }
26     return acc;
27 }
```

当矩阵很小，两个函数都在缓存里跑，差距可能不明显；当矩阵超过缓存容量，列访问就会暴露跨步访问的问题。AI 计算中的很多布局变换，例如 NHWC 和 NCHW 的选择、权重预打包、KV Cache 的排布，最终都要回到这个问题：下一次访问的数据，是否已经在离核心足够近的层级里。

3.2 TLB 和页粒度

缓存按 cache line 管数据，TLB 按页缓存虚拟地址到物理地址的翻译。访问大量离散页时，即使数据总量不算夸张，也可能因为 TLB miss 变慢。大模型推理中，权重很大，KV Cache 随序列增长，内存访问不仅要考虑缓存容量，还要考虑页表翻译和 NUMA 放置。

TLB 问题常被初学者忽略，因为代码里看不到它。一个指针加下标的表达式背后，硬件要完成地址生成、权限检查和地址翻译。若访问模式顺序，TLB 和预取器都容易工作；若访问模式随机，硬件很难提前准备。Embedding lookup、稀疏访问、哈希结构和不规则图计算通常比规则矩阵乘更容易受 TLB 和缓存 miss 影响。

3.3 AoS、SoA 与结构化数据

AoS (Array of Structures) 把一个对象的多个字段放在一起，适合按对象访问；**SoA** (Structure of Arrays) 把每个字段分别放成数组，适合 SIMD 和批量处理。AI 算子通常处理大批同类型数值，因此 SoA 或接近 SoA 的布局更常见。系统代码中也会遇到两者取舍：一个推理请求对象可能适合 AoS，但一批 token 的 logits 或概率更适合连续数组。

Listing 3.2 AoS 和 SoA 的访问形状

```
1 #include <stdint>
2 #include <vector>
3
4 struct TokenStateAos {
5     float logit;
6     float probability;
7     std::int32_t token_id;
8 };
9
10 struct TokenStateSoa {
11     std::vector<float> logits;
12     std::vector<float> probabilities;
13     std::vector<std::int32_t> token_ids;
14 };
```

如果代码要对所有 logit 做 Softmax，SoA 更容易顺序扫描，也更容易向量化。如果代码经常拿某一个 token 的全部字段做复杂逻辑，AoS 可能更自然。布局没有绝对好坏，只有访问模式是否匹配。

3.4 预取与局部性边界

硬件预取器擅长发现顺序和固定步长访问，不擅长预测复杂间接访问。手动预取有时有用，但必须谨慎。预取太晚，数据仍然来不及；预取太早，数据可能在使用前被挤出缓存；预取太多，会污染缓存并占用带宽。本册会优先强调通过布局和循环顺序创造自然局部性，而不是一开始就依赖手动预取。

核心思想

缓存优化不是把所有数据都塞进缓存，而是让即将重复使用的数据在重复使用前不要离开合适的缓存层级。分块、packing、循环交换和算子融合都是围绕这个目标展开。

习题与作业

把一个大矩阵分别按行和按列求和，记录不同矩阵规模下的运行时间。矩阵从 L1 cache 能容纳的规模开始，逐步扩大到 L2、L3 之外。观察差距从哪里开始变大，并结合 cache line 解释原因。

第 4 章

SIMD：从标量循环到向量化

SIMD 的全称是 Single Instruction Multiple Data，即一条指令处理多个数据 lane。对 AI 计算而言，SIMD 不是装饰，而是单核吞吐的核心来源。一个标量循环每次处理一个 float，向量化后一次可以处理四个、八个、十六个甚至更多元素。问题在于，数学上能并行，不代表代码一定能被编译器或手写 intrinsic 正确高效地向量化。

4.1 向量化需要独立性

最容易向量化的是逐元素循环。例如 $C_i = A_i + B_i$ ，每个元素的计算互不依赖，地址连续，循环次数清楚。编译器可以把多个元素合成一个向量加载、一个向量加法和一个向量存储。若循环中存在跨迭代依赖，例如前缀和，后一个元素依赖前一个元素的结果，向量化就困难得多。

Listing 4.1 容易向量化的逐元素循环

```
1 #include <cstdint>
2 #include <span>
3
4 void add_arrays(std::span<const float> a,
5               std::span<const float> b,
6               std::span<float> c) {
7     const std::int64_t n = static_cast<std::int64_t>(c.size());
8     for (std::int64_t i = 0; i < n; ++i) {
9         c[static_cast<std::size_t>(i)] =
10            a[static_cast<std::size_t>(i)] + b[static_cast<std::size_t>(i)];
11     }
12 }
```

这段代码看起来普通，但它具备向量化的基本条件：连续内存、固定步长、没有循环携带依赖。若把输出写到输入重叠的区域，或者通过不透明指针访问，编译器就可能因为别名风险保守。工业代码会通过接口约束、对齐信息、restrict 风格提示或库内部封装减少这种不确定性。

4.2 归约为什么更复杂

点积也是 AI 计算常见循环，但它比逐元素加法复杂，因为所有迭代都更新同一个累加器。向量化点积通常不是让多个 lane 同时写一个标量，而是维护多个部分和，最后再做水平归约。这样做可以减少依赖链长度，提高指令级并行。

Listing 4.2 教学版点积：多个累加器降低依赖链压力

```
1 #include <cstdint>
2 #include <span>
3
4 float dot_unrolled(std::span<const float> a,
5                   std::span<const float> b) {
6     const std::int64_t n = static_cast<std::int64_t>(a.size());
7     float acc0 = 0.0F;
8     float acc1 = 0.0F;
9     float acc2 = 0.0F;
10    float acc3 = 0.0F;
```



```

11
12     std::int64_t i = 0;
13     for (; i + 3 < n; i += 4) {
14         acc0 += a[static_cast<std::size_t>(i)] *
15                b[static_cast<std::size_t>(i)];
16         acc1 += a[static_cast<std::size_t>(i + 1)] *
17                b[static_cast<std::size_t>(i + 1)];
18         acc2 += a[static_cast<std::size_t>(i + 2)] *
19                b[static_cast<std::size_t>(i + 2)];
20         acc3 += a[static_cast<std::size_t>(i + 3)] *
21                b[static_cast<std::size_t>(i + 3)];
22     }
23
24     float acc = (acc0 + acc1) + (acc2 + acc3);
25     for (; i < n; ++i) {
26         acc += a[static_cast<std::size_t>(i)] *
27                b[static_cast<std::size_t>(i)];
28     }
29     return acc;
30 }

```

这不是最终的 SIMD intrinsic 版本，但它揭示了一个重要思想：归约的瓶颈不只是操作数量，还有依赖链。现代 CPU 可以同时执行多条独立浮点指令，但如果每条指令都依赖上一条累加结果，吞吐就会被延迟限制。多个累加器让硬件有更多独立工作可做。

4.3 对齐、尾部和掩码

SIMD 循环通常要处理三个细节。第一是对齐。对齐访问更容易让硬件高效工作，虽然现代 CPU 对非对齐访问的支持已经比过去好很多。第二是尾部。数组长度不一定刚好是向量宽度的整数倍，尾部元素要用标量循环、掩码加载或特殊路径处理。第三是异常和边界。若向量加载越过合法内存，即使多出来的 lane 不参与结果，也可能触发错误；因此边界处理必须明确。

AVX-512 提供更强的掩码机制，能让尾部处理更自然；AVX2 常见做法是主循环处理完整向量，尾部用标量补齐。不同平台有不同权衡。本册不会把所有 intrinsic 当成必背 API，而会先建立向量语义：一组 lane 同步执行，同一条指令处理多个元素，水平归约和跨 lane shuffle 是额外成本。

4.4 编译器自动向量化与手写内核

能自动向量化时，应先让编译器做。编译器知道目标架构、寄存器分配和调度细节，也能随编译选项适配不同平台。但矩阵乘、卷积和量化 GEMM 这类核心内核往往需要手写或生成，因为它们涉及寄存器块、数据 packing、预取、指令选择和微架构调度。手写内核不是为了炫技，而是因为通用编译器很难从普通三重循环推导出工业库需要的全部结构。

常见误区

不要把 SIMD 等同于“把循环步长改成向量宽度”。真正的问题包括别名、对齐、尾部、依赖链、数据布局、寄存器压力和内存带宽。向量化增加了单核计算能力，也可能把瓶颈更快推到内存系统。

实验建议：使用编译器的向量化报告，例如 GCC 的 `-fopt-info-vec` 或 Clang 的 `-Rpass=loop-vectorize`，观察逐元素加法、点积、前缀和三个循环分别能否自动向量化。把报告和汇编对照起来，确认是否出现向量加载和向量浮点指令。

第零部分

张量内核：矩阵、卷积与算子融合

第 5 章

矩阵乘：从三重循环到性能结构

矩阵乘是 AI 计算里最重要的内核之一。全连接层、卷积降维、Transformer 的投影、MLP、Attention 中的 QK^T 和 PV 都可以直接或间接落到 GEMM。矩阵乘的公式很简单： $C_{ij} = \sum_k A_{ik} B_{kj}$ 。困难在于，这个公式对应的数据访问和复用空间极大，朴素三重循环通常离硬件上限很远。

5.1 先看最朴素的三重循环

假设矩阵按行主序存储， A 的形状是 $M \times K$ ， B 的形状是 $K \times N$ ， C 的形状是 $M \times N$ 。最直接的写法如下。

Listing 5.1 朴素 GEMM：按 i、j、k 顺序计算

```
1 #include <stdint>
2 #include <span>
3
4 void gemm_ijk(std::span<const float> a,
5               std::span<const float> b,
6               std::span<float> c,
7               std::int32_t m,
8               std::int32_t n,
9               std::int32_t k_size) {
10     for (std::int32_t i = 0; i < m; ++i) {
11         for (std::int32_t j = 0; j < n; ++j) {
12             float acc = 0.0F;
13             for (std::int32_t k = 0; k < k_size; ++k) {
14                 const float av =
15                     a[static_cast<std::size_t>(i) * k_size + k];
16                 const float bv =
17                     b[static_cast<std::size_t>(k) * n + j];
18                 acc += av * bv;
19             }
20             c[static_cast<std::size_t>(i) * n + j] = acc;
21         }
22     }
23 }
```

这个版本对 A 的访问是顺序的，对 B 的访问是跨行跳跃的：固定 j ， k 每增加一，地址增加 N 个 float。如果 N 很大，每次访问 B 都可能落在新的 cache line 上。数学上， B 的同一行会被多个 j 使用， A 的同一个元素会被多个输出列使用，但这个循环顺序并没有很好地让缓存捕获这些复用。

5.2 循环顺序改变复用位置

把循环顺序改成 i, k, j ，可以让 B 的一整行被顺序扫描，同时让 A_{ik} 复用于多个输出列。代码如下。

Listing 5.2 按 i、k、j 顺序暴露 A 元素对一行 C 的复用

```
1 #include <algorithm>
2 #include <stdint>
```

```

3 #include <span>
4
5 void gemm_ikj(std::span<const float> a,
6             std::span<const float> b,
7             std::span<float> c,
8             std::int32_t m,
9             std::int32_t n,
10            std::int32_t k_size) {
11     std::fill(c.begin(), c.end(), 0.0F);
12     for (std::int32_t i = 0; i < m; ++i) {
13         for (std::int32_t k = 0; k < k_size; ++k) {
14             const float av =
15                 a[static_cast<std::size_t>(i) * k_size + k];
16             for (std::int32_t j = 0; j < n; ++j) {
17                 c[static_cast<std::size_t>(i) * n + j] +=
18                     av * b[static_cast<std::size_t>(k) * n + j];
19             }
20         }
21     }
22 }

```

这个版本常常比 i, j, k 顺序更友好，因为内层 j 顺序访问 B 和 C 。但它也带来新问题： C 的同一行会被反复读写 K 次。如果 N 很大， C 行不能稳定留在高层缓存里，读写流量仍然很高。矩阵乘优化不是找到一个永远最好的循环顺序，而是让 A 、 B 、 C 的工作块同时适合缓存和寄存器。

5.3 复杂度相同，常数完全不同

两个版本的时间复杂度都是 $O(MKN)$ ，但性能可能差很多。这说明大 O 只能描述增长阶，不能描述内存层级、SIMD 和常数因素。对于 AI 内核，复杂度分析仍然有用，因为它告诉我们计算量规模；但要预测真实性能，还必须把地址序列和复用结构写出来。

矩阵乘有一个重要性质：每个 A_{ik} 会参与 N 个输出，每个 B_{kj} 会参与 M 个输出。如果能把一小块 A 和一小块 B 留在缓存或寄存器里，它们就能产生一小块 C 的许多结果。这就是分块的基本动机。后面的章节会看到，工业 GEMM 把这个思想分成多层：L3 cache 级别的大块、L2 cache 级别的 panel、L1 cache 级别的 packed block、寄存器级别的 micro tile。

核心思想

矩阵乘快，不是因为公式神秘，而是因为它有高复用潜力。优化的核心是让这种潜力真实落到缓存、寄存器和 SIMD lane 上，而不是只停留在数学表达式里。

5.4 误差和累加顺序

浮点矩阵乘还涉及数值问题。不同循环顺序、不同分块、不同 SIMD 归约顺序，可能得到略有差异的结果，因为浮点加法不满足严格结合律。高性能库通常接受这种微小差异，并用误差容忍检查正确性。教材实验中不要用完全相等比较 float，而应使用绝对误差和相对误差。

Listing 5.3 浮点结果检查：同时看绝对误差和相对误差

```

1 #include <cmath>
2
3 bool close_enough(float actual, float expected) {
4     const float abs_error = std::fabs(actual - expected);
5     const float scale = std::max(1.0F, std::fabs(expected));
6     return abs_error <= 1.0e-4F * scale;
7 }

```

习题与作业

实现 i, j, k 和 i, k, j 两个版本，使用相同输入验证结果接近。随后固定 M 和 K ，逐步增加 N ，观察两个版本的差距如何变化。把观察结果和 B 、 C 的访问轨迹联系起来解释。

第 6 章

分块、Packing 与微内核

上一章看到，改变循环顺序可以改善一部分访问模式，但还不够。真正高性能的 GEMM 往往会分成多层结构：先把矩阵切成块，再把块复制成更适合内核访问的 packed 格式，最后用一个手写或生成的微内核在寄存器里计算小块 C 。这一章解释这些结构为什么存在。

6.1 分块的根本目的

分块的目标是控制工作集大小。假设我们计算 C 的一个小块，它需要 A 的若干行和 B 的若干列。只要这部分 A 、 B 、 C 能留在合适缓存层级里，内层计算就能反复复用它们。若块太大，缓存装不下，数据被提前挤出；若块太小，循环开销、边界处理和 packing 成本占比过高。

可以先写一个教学版分块，不考虑最优块大小。

Listing 6.1 教学版分块 GEMM：按块组织复用

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <span>
4
5 void gemm_blocked(std::span<const float> a,
6                  std::span<const float> b,
7                  std::span<float> c,
8                  std::int32_t m,
9                  std::int32_t n,
10                 std::int32_t k_size,
11                 std::int32_t block) {
12     std::fill(c.begin(), c.end(), 0.0F);
13     for (std::int32_t ii = 0; ii < m; ii += block) {
14         for (std::int32_t kk = 0; kk < k_size; kk += block) {
15             for (std::int32_t jj = 0; jj < n; jj += block) {
16                 const std::int32_t i_end = std::min(ii + block, m);
17                 const std::int32_t k_end = std::min(kk + block, k_size);
18                 const std::int32_t j_end = std::min(jj + block, n);
19                 for (std::int32_t i = ii; i < i_end; ++i) {
20                     for (std::int32_t k = kk; k < k_end; ++k) {
21                         const float av =
22                             a[static_cast<std::size_t>(i) * k_size + k];
23                         for (std::int32_t j = jj; j < j_end; ++j) {
24                             c[static_cast<std::size_t>(i) * n + j] +=
25                                 av * b[static_cast<std::size_t>(k) * n + j];
26                         }
27                     }
28                 }
29             }
30         }
31     }
32 }
```

这段代码还不是工业实现，但它已经表达了分块思想。它不是减少总 FLOPs，而是改变数据在

缓存里的停留方式。分块参数需要按目标机器调优，因为 L1、L2、L3 容量、cache associativity、SIMD 宽度、寄存器数量和预取行为都会影响最佳值。

6.2 为什么需要 Packing

分块之后，仍然可能存在跨步访问和边界复杂性。Packing 的做法是把即将计算的一块 A 或 B 复制到连续、对齐、顺序访问的临时缓冲区。初看这像额外成本：明明原矩阵里已经有数据，为什么还要复制一次？答案是复制一次可以换来内层微内核大量规则访问。只要 packed block 被复用足够多次，复制成本就能被摊薄。

Packing 还让微内核更简单。微内核不必处理原矩阵的 leading dimension、跨步、边界和奇怪布局，只要按固定格式读取连续数据。许多库会把 B 的 panel 打包成适合连续向量加载的形状，把 A 的小块打包成适合广播或向量加载的形状。这样内层循环就能专注于 FMA 和寄存器调度。

心智模型

Packing 是一次主动的数据重排。它用可控的顺序拷贝，换取后续大量计算中的规则加载、少分支、少地址计算和更稳定的预取效果。

6.3 微内核和寄存器块

微内核通常计算 C 的一个很小 tile，例如 $M_R \times N_R$ 。这个 tile 的累加值保存在寄存器中，循环遍历 K 维，每一步从 A 和 B 取数据并执行 FMA。等 K 维完成后，再把寄存器里的 tile 写回 C 。

为什么要把 C 的小块放在寄存器里？因为 C 的每个元素要累加很多次。如果每次乘加都读写内存，流量巨大；若把部分和保存在寄存器，内层只在最后写回一次。寄存器数量决定了 tile 不能无限大，SIMD 宽度决定了 N_R 或 M_R 常常按向量 lane 组织。

Listing 6.2 教学化微内核形状：寄存器保存 2x4 的 C 小块

```

1 #include <stdint>
2 #include <span>
3
4 void microkernel_2x4(std::span<const float> a,
5                     std::span<const float> b,
6                     std::span<float> c,
7                     std::int32_t k_size,
8                     std::int32_t c_stride) {
9     float c00 = 0.0F;
10    float c01 = 0.0F;
11    float c02 = 0.0F;
12    float c03 = 0.0F;
13    float c10 = 0.0F;
14    float c11 = 0.0F;
15    float c12 = 0.0F;
16    float c13 = 0.0F;
17
18    for (std::int32_t k = 0; k < k_size; ++k) {
19        const float a0 = a[static_cast<std::size_t>(k)];
20        const float a1 = a[static_cast<std::size_t>(k_size) + k];
21        const std::size_t b_base = static_cast<std::size_t>(k) * 4U;
22        const float b0 = b[b_base + 0U];
23        const float b1 = b[b_base + 1U];
24        const float b2 = b[b_base + 2U];
25        const float b3 = b[b_base + 3U];
26        c00 += a0 * b0;
27        c01 += a0 * b1;

```

```

28     c02 += a0 * b2;
29     c03 += a0 * b3;
30     c10 += a1 * b0;
31     c11 += a1 * b1;
32     c12 += a1 * b2;
33     c13 += a1 * b3;
34 }
35
36 c[0] = c00;
37 c[1] = c01;
38 c[2] = c02;
39 c[3] = c03;
40 c[static_cast<std::size_t>(c_stride) + 0U] = c10;
41 c[static_cast<std::size_t>(c_stride) + 1U] = c11;
42 c[static_cast<std::size_t>(c_stride) + 2U] = c12;
43 c[static_cast<std::size_t>(c_stride) + 3U] = c13;
44 }

```

这段代码没有使用 `intrinsic`，但已经体现微内核思想。真正的 SIMD 版本会把一行或多行 B 装进向量寄存器，把 A 的标量广播到向量，执行向量 FMA。寄存器块越大，复用越充分，但寄存器压力也越大。超过寄存器容量后，编译器会 `spill` 到栈上，性能反而下降。

6.4 工业库的分层视角

OpenBLAS、BLIS、oneDNN 和其他库的 GEMM 结构看起来复杂，是因为它们要同时服务不同矩阵形状、不同数据类型、不同指令集和不同缓存层级。常见结构可以粗略理解为：外层按大块切分，适配 L3 和多线程；中层 `packing panel`，适配 L2；内层微内核，适配 L1、寄存器和 SIMD。源码阅读时不要一上来陷入所有宏和平台分支，先找出这三层结构。

习题与作业

阅读一个开源 BLAS 的 GEMM 路径，记录它在哪里选择 `block size`，在哪里 `packing`，在哪里调用架构相关 `microkernel`。不要求读懂所有汇编，先把数据从原矩阵到 `packed buffer` 再到 `microkernel` 的路径画出来。

第 7 章

卷积、数据流与算子融合

卷积是视觉模型中的核心算子。它的数学形式和矩阵乘不同，但在实现上常常与 GEMM 有密切关系。理解卷积性能，不能只看卷积核大小，还要看输入布局、输出布局、stride、padding、通道数、batch、缓存复用和是否需要中间展开。

7.1 卷积的循环结构

以二维卷积为例，每个输出位置会遍历输入通道和卷积核窗口。若输入形状为 $N \times C \times H \times W$ ，输出通道为 K ，卷积核为 $R \times S$ ，那么每个输出元素大约需要 CRS 次乘加。输出位置很多，权重会在不同图像位置复用，输入像素也会被相邻窗口复用。

教学版直接卷积可以写成多重循环。代码看起来长，但每一层循环都有明确含义：batch、输出通道、输出高宽、输入通道、卷积核高宽。

Listing 7.1 教学版 NCHW 直接卷积

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <span>
4
5 void conv2d_nchw_direct(std::span<const float> input,
6                         std::span<const float> weight,
7                         std::span<float> output,
8                         std::int32_t batch,
9                         std::int32_t in_channels,
10                        std::int32_t in_h,
11                        std::int32_t in_w,
12                        std::int32_t out_channels,
13                        std::int32_t kernel_h,
14                        std::int32_t kernel_w,
15                        std::int32_t out_h,
16                        std::int32_t out_w) {
17     std::fill(output.begin(), output.end(), 0.0F);
18     for (std::int32_t n = 0; n < batch; ++n) {
19         for (std::int32_t oc = 0; oc < out_channels; ++oc) {
20             for (std::int32_t oh = 0; oh < out_h; ++oh) {
21                 for (std::int32_t ow = 0; ow < out_w; ++ow) {
22                     float acc = 0.0F;
23                     for (std::int32_t ic = 0; ic < in_channels; ++ic) {
24                         for (std::int32_t kh = 0; kh < kernel_h; ++kh) {
25                             for (std::int32_t kw = 0; kw < kernel_w; ++kw) {
26                                 const std::int32_t ih = oh + kh;
27                                 const std::int32_t iw = ow + kw;
28                                 const std::size_t input_index =
29                                     ((static_cast<std::size_t>(n) * in_channels + ic) <
30                                     in_h + ih) * in_w + iw;
31                                 const std::size_t weight_index =
32                                     ((static_cast<std::size_t>(oc) * in_channels + ic) <
```

```

↪ ) *
33         kernel_h + kh) * kernel_w + kw;
34         acc += input[input_index] * weight[weight_index];
35     }
36 }
37 }
38     const std::size_t output_index =
39         ((static_cast<std::size_t>(n) * out_channels + oc) *
40          out_h + oh) * out_w + ow;
41     output[output_index] = acc;
42 }
43 }
44 }
45 }
46 }

```

这段代码没有处理 padding 和 stride，是为了突出主干。真实卷积会在边界判断、布局选择和向量化上更复杂。若每个输出位置都重新读取卷积窗口，输入复用没有充分发挥；若输出通道较多，同一个输入窗口可以同时用于多个输出通道，类似矩阵乘中 A 元素复用于多个 C 元素。

7.2 im2col: 用内存换 GEMM

一种经典方法是 **im2col** (image to column)。它把每个卷积窗口展开成一行，把卷积转化为矩阵乘。好处是可以复用高度优化的 GEMM；坏处是展开后的中间矩阵可能很大，带来额外内存流量和空间占用。im2col 的思想非常重要，因为它展示了一个常见工程取舍：把复杂访问变成规则 GEMM，但付出数据重排成本。

当 batch 大、通道多、输出位置多时，GEMM 的高效率可能覆盖 im2col 成本；当模型小、内存紧、低延迟要求高时，直接卷积或隐式 GEMM 可能更合适。工业库经常根据形状选择不同算法，而不是固定一种实现。

7.3 Winograd 和特殊卷积

对于小卷积核，尤其是 3×3 ，Winograd 算法可以减少乘法数量，但会增加加法、变换和数值误差风险。深度可分离卷积则把空间卷积和通道混合拆开，显著减少操作数，但也降低计算强度，使内存访问和布局更重要。 1×1 卷积本质上接近矩阵乘，常常直接走 GEMM 路径。

这些变种说明卷积没有单一最优路径。算法层面减少乘法，不一定总能加速；若减少计算后程序变成内存受限，收益会被带宽限制吃掉。选择卷积算法时要同时看形状、数据类型、缓存、SIMD 和误差容忍。

7.4 算子融合

卷积后面常跟 bias、BatchNorm、ReLU、残差加法等操作。若每个操作都把完整中间张量写回内存，再由下一个操作读出来，内存流量会很大。算子融合把多个操作合并到同一个内核里，例如在卷积累加完成后立即加 bias 并做 ReLU，只把最终结果写回。

Listing 7.2 卷积输出阶段融合 bias 和 ReLU 的形状

```

1 float finish_conv_value(float acc, float bias) {
2     const float with_bias = acc + bias;
3     return with_bias > 0.0F ? with_bias : 0.0F;
4 }

```

融合的境界不是越多越好。融合过多会让内核复杂、寄存器压力上升、代码生成路径爆炸，也会减少中间结果复用的灵活性。合理融合通常针对读写流量大的简单逐元素操作，让它们跟生产中间值的重算子靠在一起。

核心理想

卷积优化的主线是数据流选择：直接卷积少中间数据但访问复杂，im2col 访问规则但增加展开流量，Winograd 减乘法但增加变换和误差，融合减少中间张量但增加内核复杂度。

习题与作业

选择一个 3×3 卷积形状，估算直接卷积的 FLOPs 和输入、权重、输出字节数。再估算 im2col 展开矩阵的大小。比较二者的额外内存成本，并解释为什么库需要按形状选择算法。

第零部分

Transformer 推理：从计算图到内存流

第 8 章

归一化、Softmax 与数值稳定性

Transformer 推理里有许多看似“小”的算子：LayerNorm、RMSNorm、Softmax、激活函数、残差加法。它们的 FLOPs 可能远少于矩阵乘，但端到端性能中不能忽略，因为它们常常读写完整向量或矩阵，计算强度低，且位于关键路径上。更重要的是，这些算子对数值稳定性敏感，不能只为速度牺牲正确性。

8.1 Softmax 为什么要减最大值

Softmax 定义为 $\text{softmax}(x_i) = \exp(x_i) / \sum_j \exp(x_j)$ 。如果直接对很大的 x_i 求指数，可能溢出；如果很多值很小，可能下溢到零。常见稳定写法是先减去最大值 $m = \max_i x_i$ ，再计算 $\exp(x_i - m)$ 。这样最大指数为 1，其余不超过 1，不改变最终比例。

Listing 8.1 稳定 Softmax：先减最大值，再归一化

```
1 #include <algorithm>
2 #include <cmath>
3 #include <cstdint>
4 #include <span>
5
6 void softmax_stable(std::span<const float> input,
7                    std::span<float> output) {
8     float max_value = input[0];
9     for (std::int64_t i = 1; i < static_cast<std::int64_t>(input.size()); ++i) {
10         max_value = std::max(max_value, input[static_cast<std::size_t>(i)]);
11     }
12
13     float sum = 0.0F;
14     for (std::int64_t i = 0; i < static_cast<std::int64_t>(input.size()); ++i) {
15         const float value =
16             std::exp(input[static_cast<std::size_t>(i)] - max_value);
17         output[static_cast<std::size_t>(i)] = value;
18         sum += value;
19     }
20
21     const float inv_sum = 1.0F / sum;
22     for (std::int64_t i = 0; i < static_cast<std::int64_t>(output.size()); ++i) {
23         output[static_cast<std::size_t>(i)] *= inv_sum;
24     }
25 }
```

这个实现有三次线性扫描：求最大值，求指数和总和，归一化输出。它清楚但不是最少内存流量。优化版本会尝试融合扫描、使用向量化指数近似、按块处理长行，或者在 Attention 中避免显式写出完整 Softmax 矩阵。但无论怎么优化，“减最大值”这个稳定性原则不能丢。

8.2 归约的性能结构

Softmax、LayerNorm 和 RMSNorm 都包含归约。归约的特点是多个输入合成少量统计量，例如最大值、总和、均值、方差或平方和。归约不只是数学操作，也是一种数据依赖结构。单线程里，归约容易形成累加依赖链；多线程里，需要把局部统计量合并；SIMD 里，需要 lane 内向量运算和

lane 间水平归约。

LayerNorm 需要均值和方差。朴素实现可能先算均值，再算方差，再归一化，至少三次扫描。若向量长度很大，内存流量明显；若向量长度很小，函数调用和调度开销可能显著。高性能实现会根据 hidden size、数据类型和平台选择不同策略。

Listing 8.2 RMSNorm 的教学实现：平方和归约后缩放

```

1 #include <cmath>
2 #include <cstdint>
3 #include <span>
4
5 void rms_norm(std::span<const float> x,
6               std::span<const float> weight,
7               std::span<float> y,
8               float epsilon) {
9     float square_sum = 0.0F;
10    for (std::int64_t i = 0; i < static_cast<std::int64_t>(x.size()); ++i) {
11        const float value = x[static_cast<std::size_t>(i)];
12        square_sum += value * value;
13    }
14
15    const float mean_square =
16        square_sum / static_cast<float>(x.size());
17    const float scale = 1.0F / std::sqrt(mean_square + epsilon);
18    for (std::int64_t i = 0; i < static_cast<std::int64_t>(x.size()); ++i) {
19        y[static_cast<std::size_t>(i)] =
20            x[static_cast<std::size_t>(i)] * scale *
21            weight[static_cast<std::size_t>(i)];
22    }
23 }
```

8.3 近似函数和误差边界

指数、倒数平方根、GELU、SiLU 这类函数比加法和乘法昂贵。优化实现经常使用近似算法或硬件近似指令，然后通过误差分析保证模型质量不受明显影响。这里的关键是“误差边界”而不是“看起来差不多”。近似函数要说明最大误差、平均误差、输入范围和对后续归一化的影响。

AI 推理通常允许一定数值误差，因为训练和推理本身已经涉及浮点舍入、量化和不同后端差异。但误差不是无限的。Softmax 对极端 logits 敏感，归一化对 epsilon 和累加精度敏感，量化对离群值敏感。性能优化必须与正确性测试绑定。

常见误区

不要为了少一次扫描就丢掉稳定性。Softmax 不减最大值、方差计算不处理 epsilon、低精度累加没有误差检查，都可能在普通样例上看不出问题，却在真实模型或长序列上产生严重错误。

8.4 融合机会

归一化和逐元素激活常常适合融合。例如 RMSNorm 可以把缩放和权重乘法放在同一次输出扫描里；MLP 中的 bias、GELU 和乘法门控可以融合，减少中间张量。Softmax 在 Attention 中更特殊，如果显式产生 QK^T 的完整矩阵，再对它做 Softmax，然后再乘 V ，中间流量会非常大。FlashAttention 这类思想的核心，就是分块计算并在线维护 Softmax 统计量，避免完整中间矩阵落到慢内存。

核心思想

Transformer 中的小算子往往不是“小问题”。它们的单次 FLOPs 不高，但访问频繁、处在关键路径、涉及归约和数值稳定性。优化时要同时看内存流量和误差边界。

第 9 章

Attention、KV Cache 与内存流量

Attention 是 Transformer 的核心。推理时，它不仅包含矩阵乘，还包含 Softmax、mask、KV Cache 读写和随序列长度增长的内存访问。理解 Attention 性能，必须区分 prefill 和 decode：prefill 处理整段输入，矩阵形状较大；decode 每次生成一个 token，batch 和查询长度通常很小，但要读取历史 KV Cache。

9.1 Attention 的计算图

单头 Attention 可以写成 $O = \text{softmax}(QK^T/\sqrt{d})V$ 。多头 Attention 只是把 hidden dimension 切成多个 head 并并行计算。这个公式里有两个主要矩阵乘： QK^T 产生注意力分数，Softmax 归一化后再乘 V 得到输出。若序列长度为 S ，head dimension 为 D ，那么分数矩阵大小是 $S \times S$ ，长序列时中间矩阵非常大。

prefill 阶段， Q 、 K 、 V 都来自当前输入序列，计算类似批量矩阵乘。decode 阶段，新 token 只有一个或少量 query，但必须与历史所有 key 做点积，再对所有历史位置 Softmax，然后加权所有 value。此时计算量随历史长度线性增长，KV Cache 的读取成为关键。

9.2 KV Cache 的作用

如果每生成一个 token 都重新计算所有历史 token 的 K 和 V ，成本会非常高。KV Cache 把历史 token 的 key 和 value 保存下来，新 token 到来时只计算它自己的 Q, K, V ，然后用新 Q 与缓存中的所有历史 K 做 Attention。这样减少了重复计算，但增加了持续增长的内存占用和读取流量。

KV Cache 的布局直接影响性能。常见维度包括 layer、batch、head、sequence、head dimension。哪一维连续，决定 decode 时读取一个 head 的历史 key 是否顺序，多个 head 是否容易并行，追加新 token 是否连续写入。布局还影响量化 KV Cache、分页 KV Cache 和多请求 batching。

Listing 9.1 教学版 KV Cache 索引：明确每个维度的地址贡献

```
1 #include <cstdint>
2 #include <cstdint>
3
4 struct KvShape {
5     std::int32_t layers;
6     std::int32_t batch;
7     std::int32_t heads;
8     std::int32_t max_seq;
9     std::int32_t head_dim;
10 };
11
12 std::size_t kv_index(const KvShape& shape,
13                     std::int32_t layer,
14                     std::int32_t batch_id,
15                     std::int32_t head,
16                     std::int32_t token,
17                     std::int32_t dim) {
18     std::size_t index = static_cast<std::size_t>(layer);
19     index = index * shape.batch + static_cast<std::size_t>(batch_id);
20     index = index * shape.heads + static_cast<std::size_t>(head);
21     index = index * shape.max_seq + static_cast<std::size_t>(token);
```



```

22     index = index * shape.head_dim + static_cast<std::size_t>(dim);
23     return index;
24 }

```

这个布局让同一个 token 的 head dimension 连续。如果 decode 时要对固定 head 扫描所有历史 token，并且每个 token 读取完整 head dimension，那么 token 维和 dim 维的组合访问是规则的。但如果 batching、分页或量化改变布局，就要重新分析地址序列。

9.3 从完整矩阵到在线 Softmax

朴素 Attention 会先算完整分数矩阵，再做 Softmax，再乘 V 。这对长序列很昂贵，因为分数矩阵写入和读出都很大。在线 Softmax 的思想是按块处理分数，维护当前最大值和归一化分母。当新块到来且最大值变化时，旧的累加结果按比例缩放。这样可以不把完整分数矩阵写回内存。

这类算法说明，Attention 优化不是简单“调用更快 GEMM”。它要把矩阵乘、Softmax 和乘 V 融合成数据流问题。分块大小要适配缓存，统计量要保持数值稳定，mask 和 causal 约束要正确处理，尾部和不同序列长度也要覆盖。

9.4 decode 阶段的瓶颈

decode 阶段常见瓶颈是内存带宽和小矩阵效率。每生成一个 token，都要读取每层的 KV Cache。随着上下文变长，读取历史 KV 的字节数增长，而 query 很少，矩阵乘形状偏瘦，难以达到大 GEMM 的峰值效率。此时优化方向包括 KV Cache 量化、分组查询注意力、分页管理、连续 batching、减少中间写回和更好的线程切分。

常见误区

不要用 prefill 的吞吐推断 decode 的速度。prefill 更像大矩阵计算，decode 更像长历史缓存扫描加小矩阵计算。两者的瓶颈、批处理策略和优化手段都不同。

9.5 源码阅读入口

阅读 llama.cpp、vLLM、oneDNN Graph 或 XNNPACK 这类项目时，Attention 路径可以按四个问题定位。第一， Q, K, V 投影使用什么矩阵乘路径。第二，KV Cache 的物理布局是什么，追加和读取如何索引。第三，Softmax 是否与分数计算或乘 V 融合。第四，prefill 和 decode 是否走不同 kernel。只要这四个问题清楚，复杂代码就能分层理解。

习题与作业

给定 batch 为 1、head 数为 32、head dimension 为 128、上下文长度为 4096 的 decode 场景，估算一层读取 K 和 V 至少需要多少字节。再乘以层数，观察为什么 KV Cache 带宽会成为长上下文推理的重要问题。

第零部分

量化、并行运行时与源码阅读

第 10 章

量化与低精度计算

量化的目标，是用更少的位表示权重、激活或 KV Cache，从而减少内存占用、降低带宽压力，并使用更高吞吐的低精度指令。它不是简单地把 float 强制转换成 int8。真正的量化包含比例因子、零点、分组、校准、误差控制、低精度乘加和反量化融合。若这些环节处理不好，模型质量和性能都可能变差。

10.1 仿射量化模型

常见 int8 仿射量化把实数 x 近似为 $x \approx s(q - z)$ ，其中 q 是整数， s 是 scale， z 是 zero point。对称量化常令 $z = 0$ ，实现简单；非对称量化能更好覆盖偏移分布，但计算更复杂。权重量化通常可以离线完成，激活量化可能需要运行时统计或校准。

Listing 10.1 教学版对称 int8 量化和反量化

```
1 #include <algorithm>
2 #include <cmath>
3 #include <cstdint>
4 #include <span>
5
6 void quantize_symmetric_i8(std::span<const float> input,
7                             std::span<std::int8_t> output,
8                             float scale) {
9     const float inv_scale = 1.0F / scale;
10    for (std::int64_t i = 0; i < static_cast<std::int64_t>(input.size()); ++i) {
11        const float scaled = std::round(input[static_cast<std::size_t>(i)] *
12                                          inv_scale);
13        const float clamped = std::clamp(scaled, -127.0F, 127.0F);
14        output[static_cast<std::size_t>(i)] =
15            static_cast<std::int8_t>(clamped);
16    }
17 }
18
19 void dequantize_symmetric_i8(std::span<const std::int8_t> input,
20                              std::span<float> output,
21                              float scale) {
22    for (std::int64_t i = 0; i < static_cast<std::int64_t>(input.size()); ++i) {
23        output[static_cast<std::size_t>(i)] =
24            static_cast<float>(input[static_cast<std::size_t>(i)]) * scale;
25    }
26 }
```

scale 的选择决定误差。如果 scale 太大，小值会被压到同一个整数，精度差；如果 scale 太小，大值会饱和。常见校准方法会根据最大绝对值、百分位数或误差最小化选择范围。权重中若存在离群值，逐张量量化可能损失明显，分组量化或逐通道量化会更稳。

10.2 低精度矩阵乘

int8 矩阵乘通常把 int8 输入和权重相乘，累加到 int32，再乘 scale 得到 float 或低精度输出。这样做的原因是，多个 int8 乘积累加后范围会超过 int8。硬件可能提供点积指令，一条指令完成多

个 int8 乘法并累加到 int32。性能提升来自两个方面：数据更小，带宽压力降低；低精度点积指令吞吐更高。

但量化 GEMM 的数据流更复杂。它需要读取 scale，处理 zero point，可能需要 per-channel 或 per-group 反量化，还要决定反量化发生在内核内部还是写回后。若每个输出元素都频繁读取 scale 或做复杂校正，收益会下降。因此高性能量化内核会把反量化、bias、激活尽量融合到输出阶段。

核心思想

量化不是单纯压缩模型文件，而是改变整个性能模型。权重变小会减少带宽，低精度指令会提高计算峰值，但 scale、zero point、反量化和误差控制会引入新的成本。

10.3 bf16、fp16 与累加精度

除了 int8，CPU 上还常见 bf16 和 fp16。bf16 保留 float32 的指数位范围，尾数更短，适合深度学习中间范围大但精度要求相对可控的场景；fp16 尾数更长但指数范围更小。许多硬件支持 bf16/fp16 乘法并累加到 float32，以平衡吞吐和精度。

选择低精度格式时，要区分存储格式、计算格式和累加格式。权重可以用 int8 存储，计算时转成 int32 累加；激活可以用 bf16 存储，累加用 float32；KV Cache 可以量化存储，读取时按块反量化。不同组合对应不同误差和性能。

10.4 量化粒度

逐张量量化只用一个 scale，元数据少，计算简单，但对离群值敏感。逐通道量化为每个输出通道使用 scale，常用于权重。分组量化把一段连续权重共享一个 scale，是大语言模型权重量化常见方案。粒度越细，误差越小，但 scale 元数据越多，内核处理越复杂。

KV Cache 量化还要考虑动态增长和在线写入。每个 token、每个 head、每个 block 如何选择 scale，会影响读取效率和误差。长上下文 decode 中，KV Cache 占用巨大，量化它能显著降低内存流量，但必须验证对困惑度、生成质量和长上下文稳定性的影响。

习题与作业

取一组 float 权重，分别用逐张量 scale 和每 32 个元素一组的 scale 做 int8 量化。计算反量化后的最大绝对误差和平均绝对误差。观察离群值对逐张量量化的影响。

第 11 章

线程池、任务切分与 NUMA

单核内核写得再好，也只能使用一个核心。AI 推理要利用多核，必须把工作切成任务交给线程执行。但多线程不是免费加速。线程创建、任务分发、同步、缓存竞争、带宽饱和和 NUMA 跨节点访问都可能限制扩展。理解运行时，就是理解“哪些工作适合并行，怎样切，切到哪里，何时同步”。

11.1 任务粒度

任务太大，负载不均衡，一个线程可能长时间拖住整体；任务太小，调度和同步开销会压过计算。矩阵乘通常可以按 M 或 N 方向切块，卷积可以按 batch、输出通道或输出空间切块，Attention 可以按 head、batch 或 sequence block 切块。选择切分维度时，要看每个任务的工作量是否均衡，是否会重复读取大块数据，是否会让多个线程写同一 cache line。

线程池的基本思想是提前创建固定数量工作线程，避免每次算子调用都创建和销毁线程。任务队列保存待执行工作，工作线程取任务运行，主线程等待完成。高性能运行时还会做 work stealing、亲和性绑定、分层调度和 NUMA 感知分配。

Listing 11.1 教学版并行 for 接口形状

```
1 #include <cstdint>
2 #include <functional>
3
4 class ParallelFor {
5 public:
6     void run(std::int32_t begin,
7             std::int32_t end,
8             std::int32_t grain,
9             const std::function<void>(std::int32_t, std::int32_t)>& body);
10 };
11
12 void parallel_rows(ParallelFor& parallel_for,
13                  std::int32_t rows,
14                  const std::function<void>(std::int32_t, std::int32_t)>& body) {
15     constexpr std::int32_t ROW_GRAIN = 16;
16     parallel_for.run(0, rows, ROW_GRAIN, body);
17 }
```

这只是接口形状，不是完整线程池实现。它强调一个重要点：并行库应该让调用者表达任务范围和粒度，而不是在算子内部随意创建线程。真实实现还需要异常传播、停止机制、队列、条件变量或无锁结构、线程生命周期和亲和性策略。

11.2 带宽饱和

有些内核单线程就接近内存带宽上限，多线程只会更快把带宽打满，超过某个线程数后不再加速。逐元素操作、拷贝、量化转换、某些 Softmax 和 KV Cache 扫描都可能出现这种情况。此时增加线程不但无益，还可能因为缓存争用和调度开销变慢。

判断带宽饱和要结合模型和测量。若每个元素只做很少计算，计算强度低，且多线程速度在某个线程数后平台化，就要怀疑带宽限制。可以用 STREAM 类 benchmark 估算机器内存带宽上限，再看内核实际带宽占比。若已经接近上限，下一步优化应减少字节数或融合算子，而不是继续增加线程。

11.3 False Sharing 和写入冲突

多个线程写不同变量，也可能互相影响。如果这些变量落在同一个 cache line 上，硬件缓存一致性协议会让 cache line 在核心之间来回转移，这就是 false sharing。并行写输出数组时，按连续大块切分通常比交错切分更安全。线程本地统计量也应避免紧挨在同一 cache line 上。

矩阵乘的并行切分还要避免多个线程同时更新同一块 C 。若按 K 维切分，每个线程计算部分和，最后需要归约，写冲突和额外内存都要处理；若按 M 或 N 块切分，每个线程负责独立输出块，通常更简单。不同形状下选择会变化，但原则是让写入所有权清晰。

11.4 NUMA 的影响

多路服务器或某些高端平台有 NUMA 结构：不同 CPU socket 或内存节点访问本地内存和远端内存的延迟、带宽不同。若线程在节点 0 上运行，却频繁访问节点 1 分配的内存，性能会下降。大模型权重和 KV Cache 占用巨大，NUMA 放置会影响端到端吞吐。

NUMA 优化包括首次触碰策略、线程绑定、按节点复制只读权重、按节点切分请求、避免跨节点频繁写共享数据等。是否值得做 NUMA 优化取决于部署环境。单手机或单 socket 机器上无需过早复杂化；多 socket 推理服务器上，NUMA 往往是必须解释的问题。

常见误区

多线程扩展不是线性的。若一个内核从 1 到 4 线程加速明显，从 4 到 8 线程停滞，不要只怀疑线程池实现。先判断是否已经受内存带宽、LLC 容量、写冲突或 NUMA 限制。

习题与作业

对同一个向量加法、矩阵乘和 Softmax 分别测试 1、2、4、8 个线程。记录加速比，而不是只记录时间。解释哪个内核扩展最好，哪个最早平台化，并把原因写成计算强度和内存流量分析。

第 12 章

源码阅读、Benchmark Suite 与工程报告

前面章节讲的是原理和局部内核。真正进入工程时，读者会面对复杂代码库：平台宏、模板、代码生成、线程池、内存池、图优化、量化格式和大量 benchmark。源码阅读不能靠从第一行读到最后一行，而要带着问题建立地图。本章给出一套阅读和验证方法。

12.1 先找数据结构

高性能 AI 项目的核心通常不是某个孤立函数，而是一组数据结构。矩阵或张量如何描述 shape、stride、dtype、layout；权重如何打包；线程任务如何表示；KV Cache 如何分页；量化 block 如何保存 scale；benchmark 如何描述输入形状。这些结构决定后续函数能做什么。

阅读 oneDNN 时，可以先找 primitive descriptor、memory descriptor 和具体 ISA kernel 的关系。阅读 OpenBLAS 或 BLIS 时，可以先找 GEMM 的 block size、packing buffer 和 microkernel 表。阅读 XNNPACK 时，可以先找 operator、microkernel、indirection buffer 和 threadpool。阅读 llama.cpp 时，可以先找 tensor、ggml graph、backend、KV cache 和量化 block。

12.2 再找热路径

热路径是实际运行中占主要时间的代码。可以通过 profiler 找，也可以从模型结构推断。大语言模型推理中，矩阵乘、Attention、RMSNorm、RoPE、采样和 KV Cache 操作都可能在热路径上。读源码时要区分热路径和配置路径。配置路径复杂但只执行一次，热路径哪怕一行也可能影响吞吐。

阅读热路径时建议画出数据流：输入张量从哪里来，经过哪个 layout，是否 packing，进入哪个 kernel，输出写到哪里，是否立即被下一个算子读取。每个边都标注 dtype、shape 和内存位置。这样做比单纯抄函数调用栈更有用，因为性能瓶颈通常发生在边上，也就是数据移动处。

12.3 Benchmark Suite 的结构

单个 benchmark 容易片面。一个工程化 benchmark suite 应该覆盖多个层级：微基准测单个循环或内核，算子基准测 GEMM、卷积、Softmax、Attention，模型片段基准测一层 Transformer，端到端基准测 prefill、decode、tokens/s 和 latency。每个层级回答不同问题。

基准测试还要覆盖形状。只测一个大矩阵会让 GEMM 看起来很好，却无法解释小 batch decode；只测短序列无法解释长上下文 KV Cache；只测平均值无法解释服务尾延迟。一个合理 suite 至少包含大中小矩阵、不同 batch、不同 sequence length、不同线程数、不同 dtype 和冷/热缓存场景。

Listing 12.1 Benchmark case 描述应显式记录形状和线程数

```
1 #include <stdint>
2 #include <string>
3
4 struct BenchmarkCase {
5     std::string name;
6     std::int32_t batch;
7     std::int32_t sequence_length;
8     std::int32_t hidden_size;
9     std::int32_t heads;
10    std::int32_t head_dim;
11    std::int32_t threads;
12 };
```

记录结构化 case 的好处是，结果可以长期比较。后续如果改了 packing、线程切分或量化格式，

可以直接看哪些形状变快，哪些形状变慢。没有结构化 case，benchmark 只是一堆难以追溯的数字。

12.4 工程报告怎么写

性能报告应当先写结论，再写证据。结论包括优化目标、变更范围、主要收益和风险。证据包括硬件环境、编译选项、输入形状、测量方法、统计方式、性能表、计数器和正确性验证。报告还要写失败尝试，因为失败尝试能说明边界，避免后续重复踩坑。

一份合格报告不应该只说“GEMM 优化后快了 30%”。它要说明在哪个 CPU、什么矩阵形状、什么线程数、什么 dtype、基线是谁、重复几次、误差如何、是否影响其他形状、为什么快。若推测原因是 cache miss 降低，就给出 perf 证据；若推测原因是向量化成功，就给出汇编或编译器报告。

核心思想

源码阅读、benchmark 和报告是一件事的三个面：源码给出机制，benchmark 给出事实，报告把机制和事实连接成可复查的工程结论。

12.5 后续扩写方向

本册后续可以继续扩写四类材料。第一类是更完整的实验代码，包括 GEMM 分块、AVX2 点积、Softmax 优化、int8 量化和线程池。第二类是源码专题，逐步拆解 OpenBLAS、oneDNN、XNNPACK、llama.cpp 和 Linux 调度/内存相关代码。第三类是模型案例，例如 MLP、Attention、KV Cache 长上下文和端到端 decode。第四类是报告模板和性能回归工具，让每次优化都能留下证据。

习题与作业

选择一个开源项目中的 GEMM 或 Attention benchmark，整理它的输入形状、线程设置、计时范围和正确性检查。指出它能回答什么问题，不能回答什么问题，并提出三个应该补充的 case。

附录 A

实验路线

第二册的实验应当随着正文逐步补齐。第一组实验是测量基础，包括向量加法、点积、矩阵乘三种内核的时间、FLOPs、字节数和 perf 计数器。第二组实验是内存层级，包括行访问和列访问、AoS 和 SoA、不同矩阵规模下的缓存行为。第三组实验是 SIMD，包括自动向量化报告、点积多累加器、尾部处理和汇编检查。

第四组实验是 GEMM，从 i, j, k 朴素版本、 i, k, j 版本、分块版本、packing 版本逐步推进，再对比 OpenBLAS 或 oneDNN 的性能。第五组实验是 Transformer 小算子，包括稳定 Softmax、RMSNorm、RoPE、Attention 的 prefill 和 decode。第六组实验是量化，包括 int8 对称量化、分组 scale、误差统计和量化矩阵乘。第七组实验是运行时，包括线程数扩展、任务粒度、false sharing 和 NUMA 观察。

每个实验必须记录环境、输入、命令、输出、正确性检查和结论。性能数据没有上下文就不能进入正文。若实验受手机、Termux 或虚拟化环境限制，也要明确记录限制。

附录 B

源码阅读索引

本附录保存后续源码专题的入口。

OpenBLAS 和 BLIS 用于阅读 GEMM 的经典分层结构：block size、packing、microkernel、架构分发和多线程切分。阅读时优先找 SGEMM 路径，不要一开始陷入所有数据类型。

oneDNN 用于阅读工业深度学习后端：memory descriptor、primitive、post-op 融合、ISA 选择、卷积和矩阵乘实现。阅读时重点观察 layout 和 post-op 如何影响内核选择。

XNNPACK 用于阅读移动端和 CPU 推理内核：operator 创建、microkernel 表、indirection buffer、threadpool 和量化路径。它适合理解小算子和低延迟场景。

llama.cpp 和 ggml 用于阅读大语言模型 CPU 推理：tensor 表示、计算图、后端分发、量化 block、KV Cache、RMSNorm、RoPE、Attention 和采样。阅读时应区分模型加载、图构建、单 token decode 和 benchmark 路径。

Linux 内核源码用于理解调度、NUMA、页表、透明大页、perf 事件和内存分配。第二册不会把 Linux 内核作为主线，但性能工程遇到线程绑定、页错误、NUMA 和 perf 计数器时，需要知道这些机制来自哪里。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核数值吞吐的重要来源。

分块 把大问题切成适合缓存或寄存器的小块，以提高数据复用。

Packing 把矩阵块复制成连续、对齐、适合微内核访问的临时布局。

微内核 GEMM 或卷积中最内层的架构相关计算核心，通常直接围绕寄存器和 SIMD 指令设计。

KV Cache Transformer decode 阶段保存历史 key 和 value 的缓存，用于避免重复计算历史 token。

量化 用低位宽整数或低精度浮点表示权重、激活或缓存，并通过 scale、zero point 等元数据近似原始实数。

NUMA 非统一内存访问结构，不同核心访问不同内存节点的成本不同。

算子融合 把多个相邻算子合并执行，减少中间张量读写和调度开销。

索引

AoS, 9

im2col, 20

SoA, 9

计算强度, 5